04

MGU BCA - 3RD SEM

Database Management Systems

Module 4 - NoSQL Databases and Big Data

By Aben Sojan

Exam - 6 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

TOPIC 1

Introduction to NoSQL

1

NoSQL means "Not only SQL": databases built for huge, fast-changing data.

Traditional SQL struggles with unstructured data like posts, videos and logs.

NoSQL systems scale horizontally by adding more servers.

They trade some strict consistency for speed and flexibility.

QUESTION BANK

1 What is NoSQL? Why was it introduced?

IMPORTANT

NoSQL stands for Not Only SQL. It is a family of database systems built to handle very large, fast-changing and unstructured data that traditional relational databases struggle with.

It was introduced because web and mobile apps like social media, e-commerce and streaming started generating data in volumes and forms (posts, videos, logs, clicks) that a single-server SQL database could not store or process fast enough.

NoSQL databases scale horizontally, which means they add more cheap servers to share the load instead of buying one bigger server. This is called scaling out.

They use flexible schemas, so documents or records can have different fields without redesigning tables first.

To gain speed and availability they usually trade off the strict ACID consistency of SQL for a looser model called BASE.

Common examples are MongoDB (documents), Cassandra (column families), Redis (key-value) and Neo4j (graph).

2 **Give two differences between SQL and NoSQL databases.**

IMPORTANT

SQL and NoSQL databases differ in four main ways: data organisation, schema, scaling and consistency.

First, data organisation: SQL stores data in fixed-schema tables of rows and columns, with relationships handled through joins, while NoSQL uses flexible models like documents, key-value pairs, column families or graphs.

Second, schema: in SQL the schema must be defined upfront and changing it later with ALTER TABLE is costly, whereas NoSQL is schema-free so each record can carry its own fields and the structure evolves freely.

Third, scaling: SQL scales vertically by moving to a bigger single server, which is expensive and has hard limits, while NoSQL scales horizontally by adding cheap servers, which handles massive data and traffic.

Fourth, consistency: SQL guarantees ACID, so every transaction leaves the data fully correct at once, while NoSQL follows BASE and is eventually consistent, trading instant correctness for speed and availability.

Fifth, query style: SQL uses the standard SQL language everywhere, while each NoSQL system has its own API suited to its data model.

Example: use SQL for bank accounts where exact balances matter, and use NoSQL for a social media feed where speed and scale matter more than instant consistency.

In short, choose SQL when correctness is non-negotiable and NoSQL when huge scale, flexibility and uptime come first.

TOPIC 2

NoSQL Data Models

2

Document model: stores JSON-like documents, e.g. MongoDB.

Key-value model: simple pairs of keys and values, e.g. Redis.

Column-family model: data grouped into column families, e.g. Cassandra.

Graph model: data as nodes and edges, e.g. Neo4j.

QUESTION BANK

3 Explain the four NoSQL data models with one example each. (Model paper)

IMPORTANT

NoSQL systems organise data in four main models, and each model fits a different kind of problem.

1. Document model: data is stored as flexible JSON-like documents, each with its own fields, and related data can be nested inside one document. Example: a product document {name: 'shoe', price: 999, reviews: [...]}. Used by MongoDB for catalogs, profiles and content management.

2. Key-value model: every key maps to exactly one value, like a giant dictionary, giving extremely fast lookups by key. Example: key 'session:1234' maps to a user's cart data. Used by Redis for caching, sessions and shopping carts.

3. Column-family model: data is stored in rows made of grouped columns called column families, which is efficient for huge sparse tables where most cells are empty. Example: a user row with column families profile: and activity:. Used by Cassandra for time-series and sensor data.

4. Graph model: data is stored as nodes and edges, so relationships are first-class citizens and traversals are fast. Example: node 'Aben' -FRIENDS_WITH-> node 'Ashik'. Used by Neo4j for social networks and fraud detection.

The rule for choosing: match the model to the shape of your data and the queries you will run most often.

4 When would you choose a graph database over a document database?

IMPORTANT

Choose a graph database over a document database when the relationships between items matter more than the items themselves.

In a graph database, connections are stored directly as edges, so queries like 'friends of friends' or 'shortest path' are single fast traversals. Examples are social networks, fraud detection tracing suspicious money paths, and recommendation engines.

In a document database, each record is self-contained and flexible, which is ideal when you mostly fetch whole records by key. Examples are product catalogs, user profiles and blog posts, where each record has different fields and nested data.

The practical difference: finding friends-of-friends in a graph is one traversal, while in a document store it needs multiple lookups or application-level joins, which gets slow as connections grow.

So the rule of thumb is: graphs for connected data and traversal queries, documents for

independent flexible records.

TOPIC 3

3

Document Databases: Uses and Features

Store data as flexible JSON-style documents; each document can have different fields.

No fixed schema: new fields can be added without rebuilding tables.

Good for catalogs, user profiles, blogs and content management.

Documents can be nested, so related data stays together in one read.

QUESTION BANK

5 **What are the features and uses of NoSQL document databases?**

IMPORTANT

Document databases store data as flexible JSON-style documents grouped into collections, much like rows grouped into tables in SQL.

Flexible schema: documents in one collection can have different fields, so you can add a new field to new documents without rebuilding anything.

Nested data: related details can live inside one document, for example a product document that contains its reviews. This keeps related data together and removes the need for many joins.

Horizontal scaling: document databases spread data across many servers, so they handle large applications and growing traffic well.

Developer friendly: JSON matches the data formats used in web and mobile apps, so moving data between the app and the database is easy.

Uses: product catalogs, user profiles, content management systems, blogging platforms and real-time analytics, anywhere records vary in shape and change often.

6 **Why are document databases a good fit for an online shopping site?**

IMPORTANT

Document databases fit online shopping sites because a store's data is varied, nested and constantly changing.

Products differ in attributes: a T-shirt has sizes and colours while a book has an author and page count. A document store lets each product keep its own fields with no rigid table schema forcing empty columns.

Nested documents keep everything together: a product document can contain its reviews, images and stock details inside it, so one read loads the whole product page fast.

Schema changes are painless: adding a new field like 'discount' to new products needs no table rebuild and does not disturb the millions of existing documents.

Scaling out handles traffic: during festival sales, more servers are added to carry the load, which document stores like MongoDB are built for.

JSON documents also match web and mobile apps naturally, since APIs already speak JSON, so less conversion code is needed.

In short, a document database matches exactly how online stores evolve: varied products, fast page reads and constant change.

TOPIC 4

CAP Theorem

4

CAP: Consistency, Availability, Partition tolerance; a distributed system can guarantee only two.

Consistency: every read gets the latest write.

Availability: every request gets a response, even during failures.

Partition tolerance: the system keeps working despite network splits.

Since partitions happen, designers choose CP or AP.

QUESTION BANK

7 Explain the CAP theorem.

IMPORTANT

The CAP theorem states that a distributed database system can fully guarantee only two of three properties at the same time: Consistency, Availability and Partition tolerance.

C, Consistency: every read receives the most recent write. All nodes see the same data at the same time.

A, Availability: every request gets a response, even when some servers have failed. The system never refuses to answer.

P, Partition tolerance: the system keeps working even when the network splits and servers cannot talk to each other. Partitions do happen in real networks.

Since network failures are unavoidable, partition tolerance cannot really be given up, so designers choose between the remaining two.

This gives two practical types: CP systems like HBase and MongoDB (when configured strictly) pick consistency over availability, and AP systems like Cassandra pick availability over strict consistency.

Example: banks usually pick CP so balances are always exact, while social feeds pick AP so the feed always loads even if some updates arrive late.

8 In the CAP theorem, why must partition tolerance be kept?

IMPORTANT

In the CAP theorem, partition tolerance must be kept because network partitions, where some servers cannot reach others, are unavoidable in any real distributed system.

Cables fail, routers crash, switches misbehave and whole cloud regions get isolated. No network is perfect, and designing as if partitions never happen is unrealistic.

Giving up partition tolerance would mean the entire system stops working the moment any network split occurs, which is unacceptable for a service that must stay up.

Therefore designers accept P as mandatory and make the real choice between consistency and availability.

That choice is guided by the application: banking systems keep consistency (CP) because showing a wrong balance is costly, while social feeds keep availability (AP) because showing a slightly stale post is harmless.

So partition tolerance is not a preference but a fact of distributed life, and the C-versus-A trade-off is where the real design decision lies.

TOPIC 5

BASE vs ACID

5

BASE: Basically Available, Soft state, Eventually consistent.

ACID prioritises correctness; BASE prioritises availability and speed.

SQL databases follow ACID; most NoSQL databases follow BASE.

Eventually consistent means all copies agree after some time, not instantly.

QUESTION BANK

9 **Distinguish between ACID and BASE.**

IMPORTANT

ACID and BASE are two different philosophies for keeping data correct, used by SQL and NoSQL databases respectively.

ACID stands for Atomicity, Consistency, Isolation and Durability. Atomicity means a transaction is all or nothing, never half done. Consistency means every transaction takes the database from one valid state to another. Isolation means concurrent transactions do not interfere with each other. Durability means once committed, data survives crashes.

BASE stands for Basically Available, Soft state and Eventually consistent. Basically Available means the system keeps responding even during failures. Soft state means data may change over time without new input while copies converge. Eventually consistent means all copies will agree after some delay.

Point by point: ACID demands instant consistency while BASE accepts temporary differences; ACID may refuse a write to stay correct while BASE accepts the write to stay available; ACID suits single strong servers while BASE suits huge distributed clusters.

Use ACID where exact correctness is non-negotiable, like banking and ticket booking, and BASE where massive scale and uptime matter more, like feeds, carts and catalogs.

10 **What does "eventually consistent" mean? Give an example.**

IMPORTANT

Eventually consistent means that after a write, all copies of the data will agree with each other after some delay, but not instantly.

In a distributed system the same data lives on many servers. A write reaches one server first and then spreads to the others through replication.

During that short window, different users can see different values for the same item. This temporary disagreement is the 'soft state' in BASE.

Example: you like a post on your phone and the count shows 101 immediately, while your friend on another server still sees 100 for a few seconds, and then both settle at 101.

This trade-off is deliberate: it keeps huge systems fast and available instead of making every user wait for every copy to update.

It is acceptable for feeds, likes and catalogs where a few seconds of staleness harms nobody, but not for banking where balances must be exact.

TOPIC 6

CRUD Operations and MongoDB Operators

6

CRUD: Create, Read, Update, Delete, the four basic data operations.

MongoDB create: db.students.insertOne({name:"Aben", marks:85}).

MongoDB read: db.students.find({marks: {$gt: 80}}).

MongoDB update: db.students.updateOne({name:"Aben"}, {$set:{marks:90}}).

MongoDB delete: db.students.deleteOne({name:"Aben"}).

Common operators: $gt, $lt, $eq, $in, $set.

QUESTION BANK

11 Explain CRUD operations in MongoDB with examples.

IMPORTANT

CRUD stands for Create, Read, Update and Delete, the four basic operations every database supports. In MongoDB each is done with a simple command on a collection.

Create: add a new document to a collection. Example: db.students.insertOne({name: 'Aben', marks: 85}) creates one student document.

Read: fetch documents that match a condition. Example: db.students.find({marks: {$gt: 80}}) returns all students whose marks are greater than 80, where $gt is the greater-than operator.

Update: change fields of matching documents. Example: db.students.updateOne({name: 'Aben'}, {$set: {marks: 90}}) sets Aben's marks to 90, where $set is the set-field operator.

Delete: remove matching documents. Example: db.students.deleteOne({name: 'Aben'}) removes Aben's document.

Other useful operators: $lt for less than, $eq for equal, $in to match any value in a list, and $gte/$lte for greater/less than or equal. Together CRUD and these operators cover nearly all everyday database work.

12 Write a MongoDB query to find students with marks between 60 and 90.

IMPORTANT

To find students with marks between 60 and 90, combine the $gte (greater than or equal) and $lte (less than or equal) operators in one find query.

```
db.students.find({marks: {$gte: 60, $lte: 90}});
```

How it works: find() searches the students collection and returns every document matching the filter.

$gte: 60 keeps only documents with marks 60 or above, and $lte: 90 keeps only documents with marks 90 or below, so together they form the 60-to-90 range.

Both conditions apply to the same marks field inside one document filter, which is why they sit together in {marks: {...}}.

Example result: if Aben has 85 and Ashik has 55, only Aben's document is returned.

Change the two numbers to get any range, and add .pretty() at the end for readable output.

13 Write about update operators in MongoDB. (Model paper)

IMPORTANT

Update operators modify fields inside MongoDB documents without rewriting the whole document.

$set changes a field's value:

```
db.students.updateOne({rollno: 101}, {$set: {marks: 95}});
```

$inc increments a number:

```
db.students.updateOne({rollno: 101}, {$inc: {marks: 5}});
```

$unset removes a field; $push appends a value to an array field.

Update operators are used with updateOne, updateMany or replaceOne, and they make MongoDB updates precise and fast.

TOPIC 7

7

Big Data Technologies: Hadoop, MongoDB, Cassandra

Hadoop: open-source platform that stores and processes huge data across clusters.

HDFS spreads files over many machines; MapReduce processes them in parallel.

MongoDB: document database used as a flexible store for big varied data.

Cassandra: column-family store built for high availability and massive write loads.

QUESTION BANK

14 Write short notes on Hadoop, MongoDB and Cassandra.

IMPORTANT

Hadoop, MongoDB and Cassandra are three pillars of big data, each solving a different part of the problem.

Hadoop is an open-source platform for storing and processing huge data across clusters of ordinary machines. HDFS (Hadoop Distributed File System) splits files into blocks spread over many machines with replication for safety, and MapReduce processes the blocks in parallel. Example: analysing one year of web server logs.

MongoDB is a schema-free document database that stores big varied data as flexible JSON-like documents. Example: storing product catalogs or user profiles where every record has different fields.

Cassandra is a distributed column-family database built for high availability and massive write

loads, with no single point of failure. Example: storing sensor readings or chat message inboxes arriving thousands per second.

Together they cover the three big-data needs: Hadoop for batch storage and analysis, MongoDB for flexible storage, and Cassandra for always-on heavy writes.

15 A company gets one crore sensor readings a day. Which technology fits and why?

IMPORTANT

For one crore sensor readings a day, Hadoop is the best fit for the storage and analysis part.

HDFS stores the huge volume cheaply across many ordinary machines, and its replication keeps the data safe even when machines fail.

MapReduce processes all the readings in parallel, so daily reports like average temperature per hour finish quickly instead of taking all day on one machine.

If the company also needs instant lookups, Cassandra fits the live-serving part: it handles massive write loads and stays available, which suits dashboards and alerts on fresh readings.

MongoDB is less ideal here because its strength is varied flexible documents, while sensor readings are uniform time-series data.

So the practical answer is: Hadoop for batch storage and daily analysis of the one crore readings, plus Cassandra alongside it for the live serving layer.

16 Define Big Data. (Model paper)

IMPORTANT

Big Data means datasets so large, fast or varied that traditional databases cannot handle them.

It is described by the 3 Vs: Volume (terabytes to petabytes), Velocity (streaming sensor and click data), Variety (text, images, video, logs).

Example: one crore sensor readings a day, or every click on a shopping website.

Such data needs distributed systems like Hadoop and NoSQL stores like MongoDB or Cassandra.

Big Data problems appear when volume, velocity or variety breaks a single relational server.

That is why the syllabus pairs it with Hadoop, MongoDB and Cassandra - the distributed answer.